Web Development

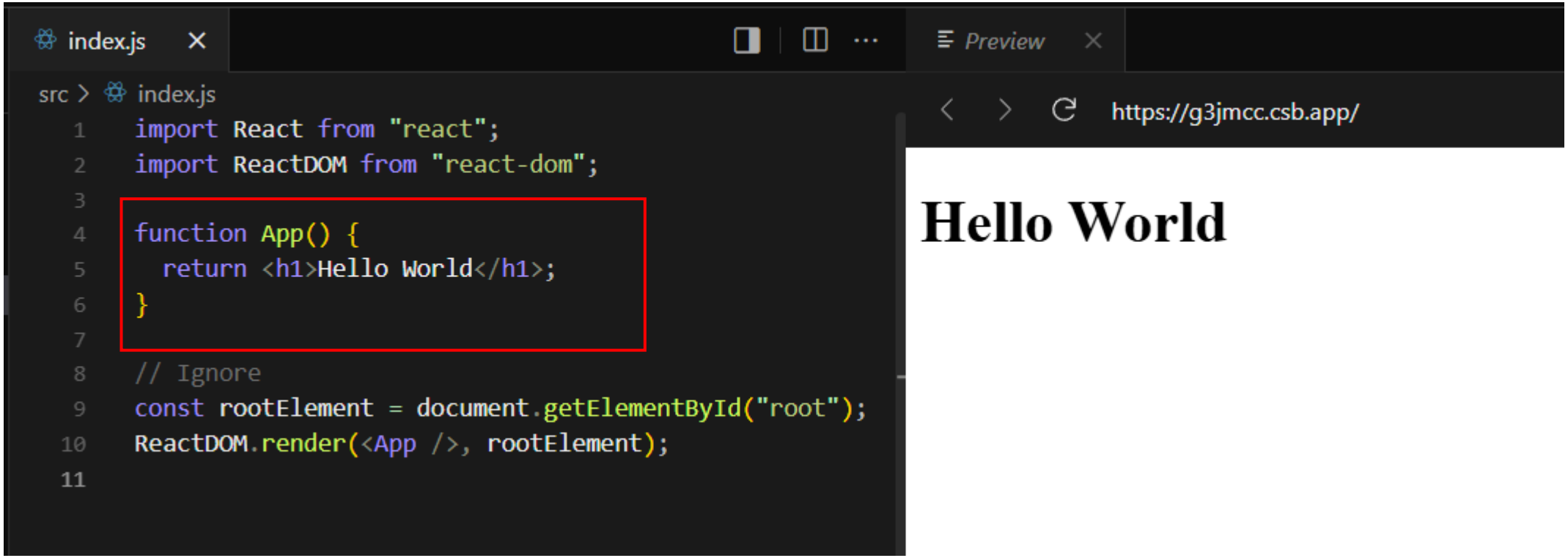
CSE343
Lab 5

Intro to React

What is React?

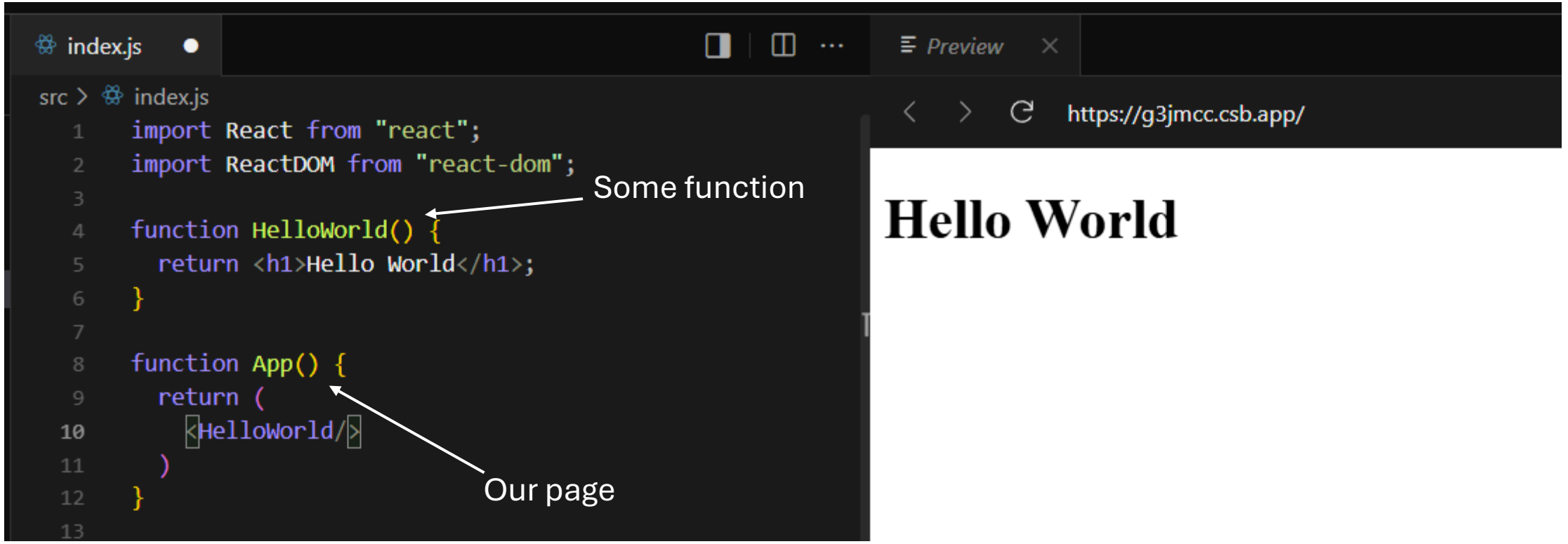
- React is a **Frontend Library** that makes frontend development easier and more scalable. How?
- Let’s discuss how after we see some React code…
- Search “react playground CodeSandbox” to start using react
- Don’t worry if you don’t understand the code. It’s just for demonstration purposes
- App.js - nodebox - CodeSandbox

Some React code



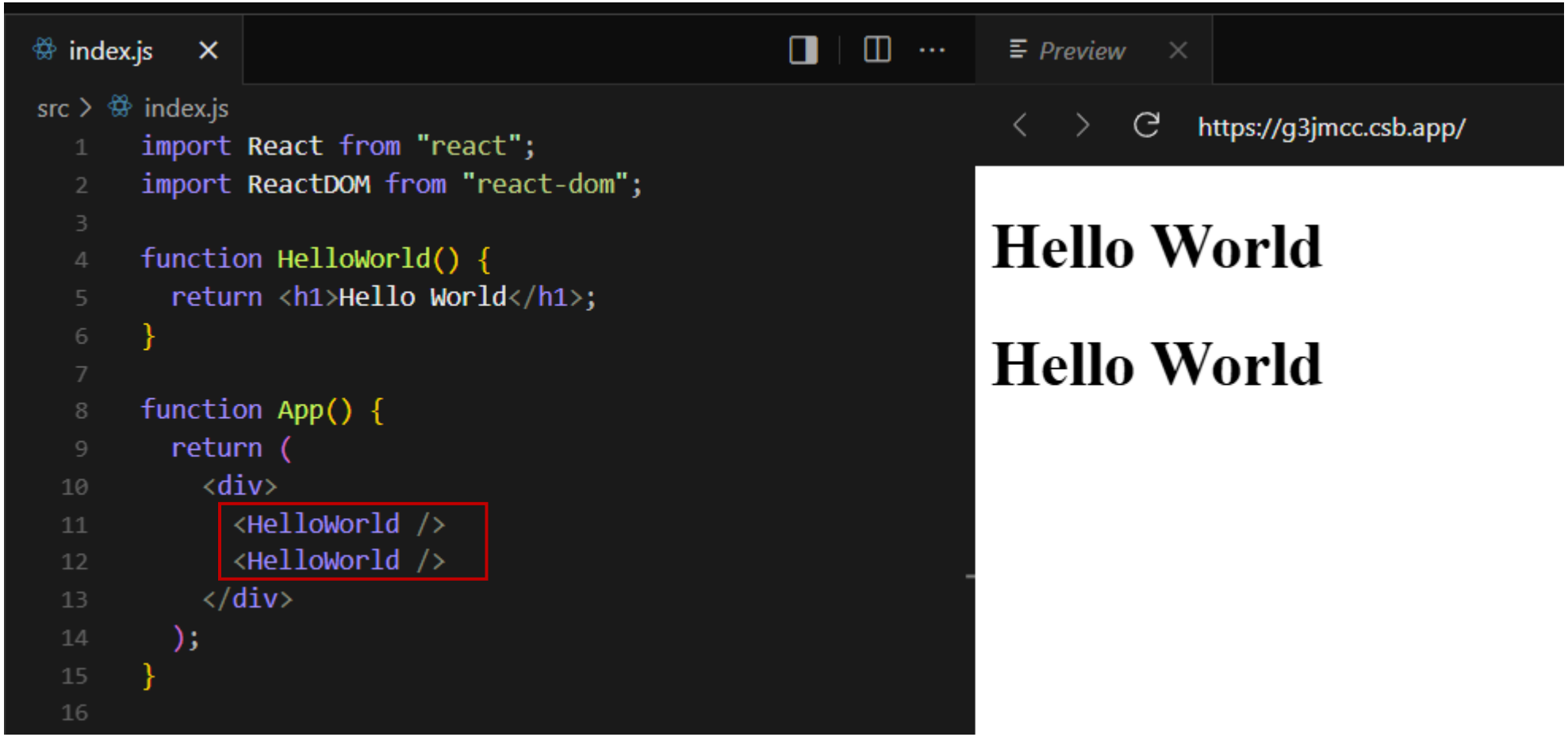
Looks familiar, doesn't it?
Let's try something else...

Some React code



So, we can write some html in some function, then use that function inside our page...
Let's try something else

Some React code



Notice how we used this HelloWorld function twice in our page

Why React

- As we saw, in react you can create some “html” in some function and reuse it elsewhere.
- That’s one powerful key feature in React where **we can write our components once and reuse** it instead of copying and pasting it.
- There are several other features in React that frontend development easier and scalable. We will explore them as we go.

Some React terms

- This function that contains html is called a component
- The language that react uses is called jsx. It looks like html but can run javascript inside it.
- We use jsx to build components and compose our UI using these components.

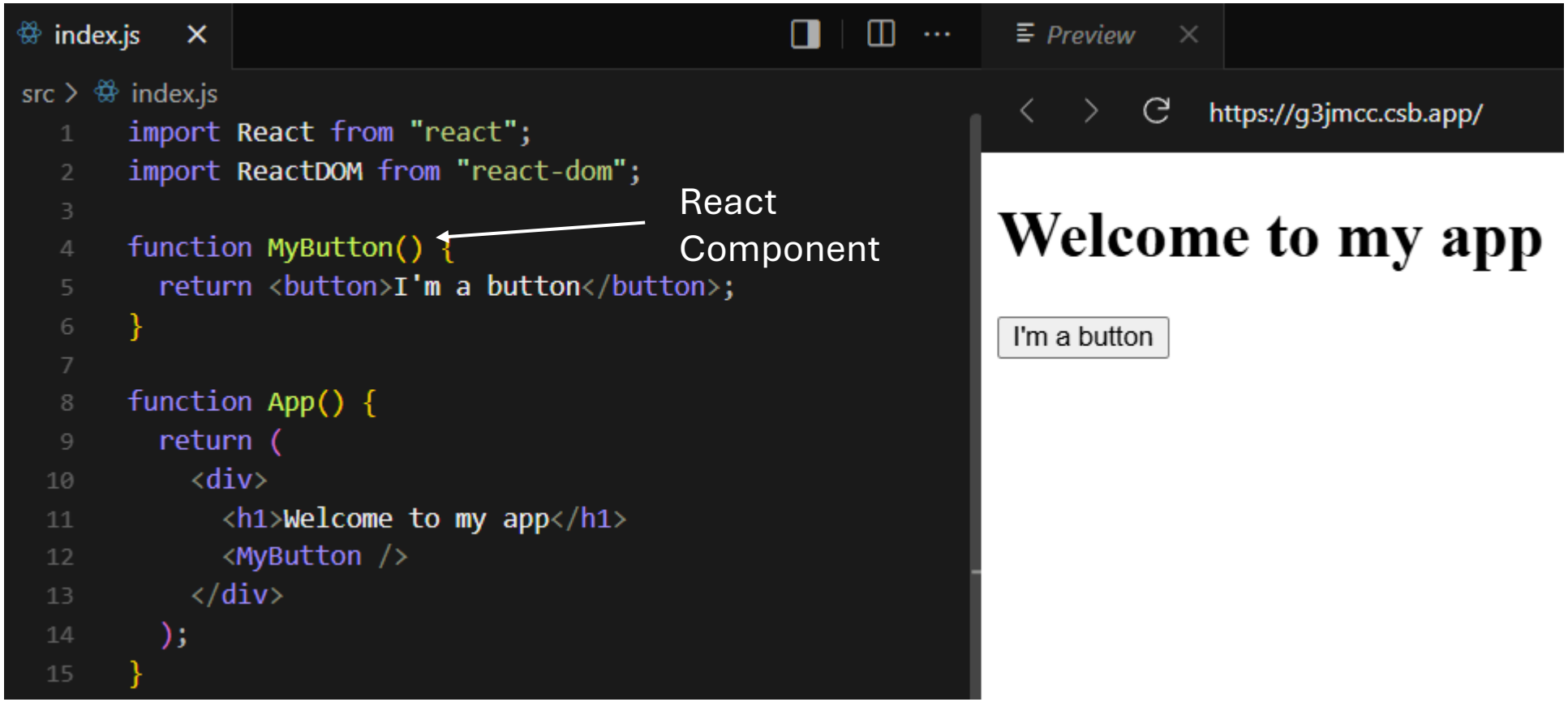
Let’s learn React

- In this lab, we will be walking through the quick start guide in react.dev

Let’s learn React – Components

- React apps are made out of *components*. A component is a piece of the UI (user interface) that has its own logic and appearance. A component can be as small as a button, or as large as an entire page.
- React components are JavaScript functions that return markup

Let’s learn React

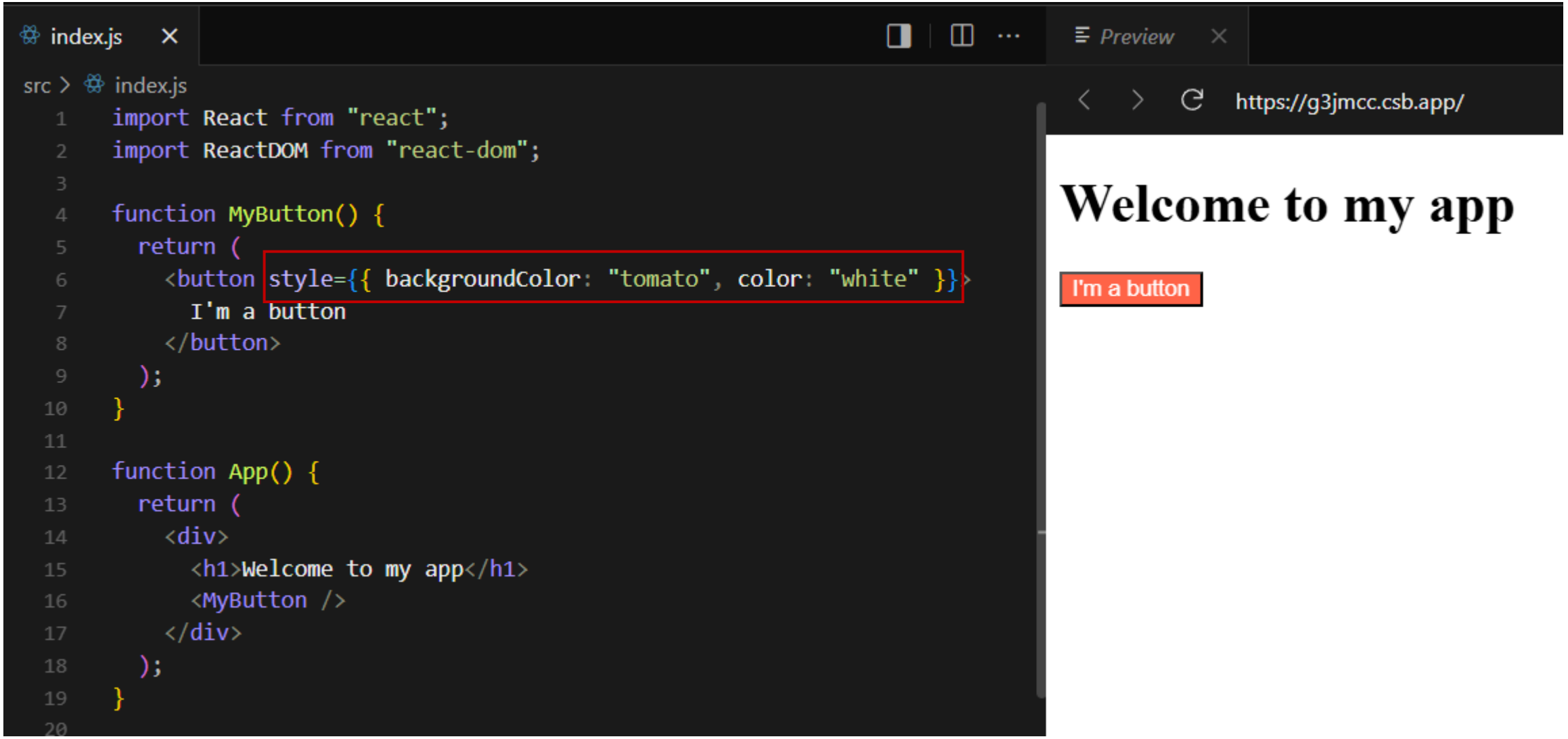


React components must start with capital letters to differentiate them from regular html tags

Let’s learn React – Styling

- React does not enforce a way to add styles to your components. You can create a .css file and import it or use inline style or use one of the many other ways to style in react.
- For the sake of simplicity, let’s use inline styles for now.

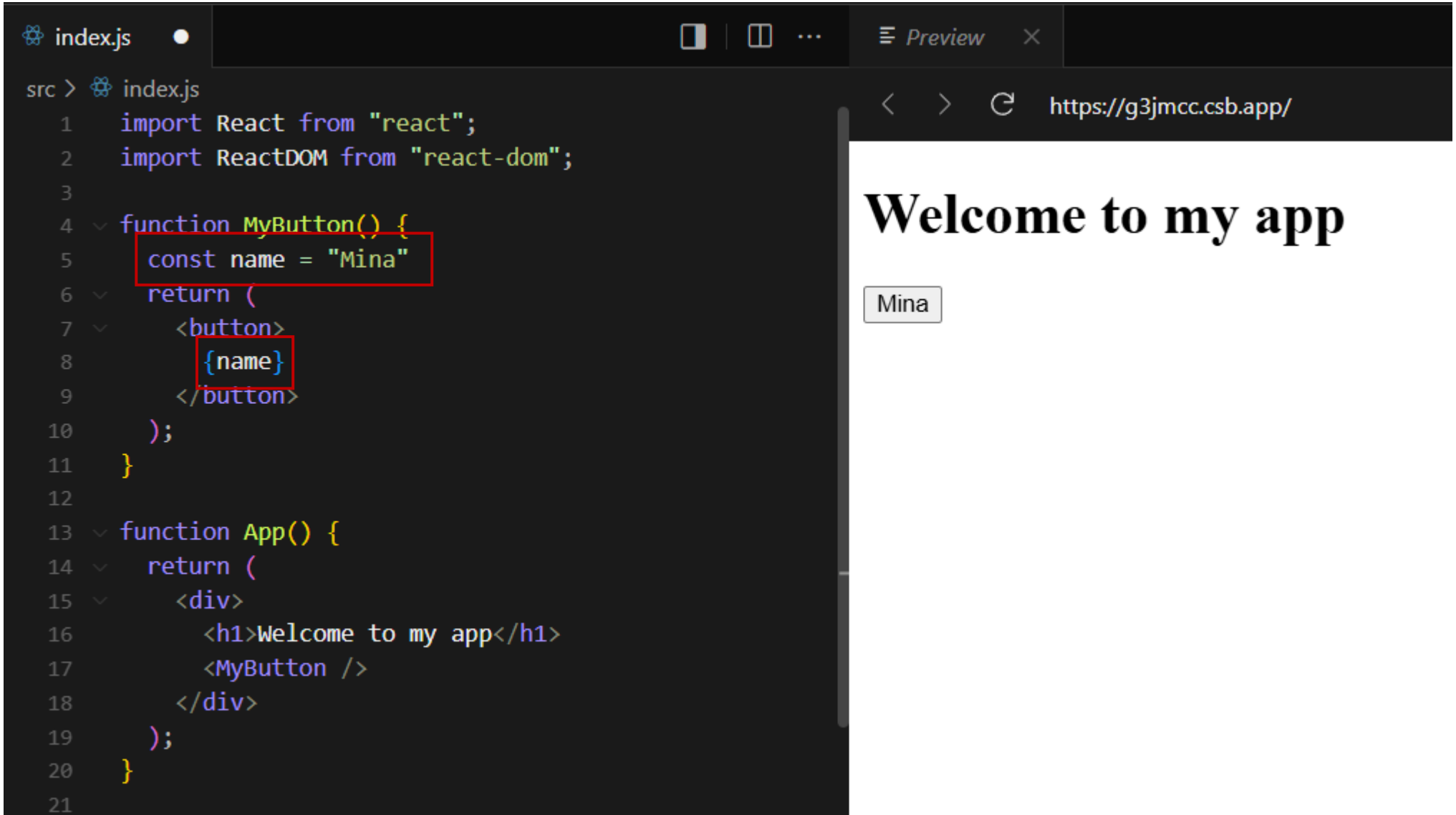
Let’s learn React



Let’s learn React – Displaying Data

- Inside your markup section, you can run javascript using curly braces

Let’s learn React – Displaying Data



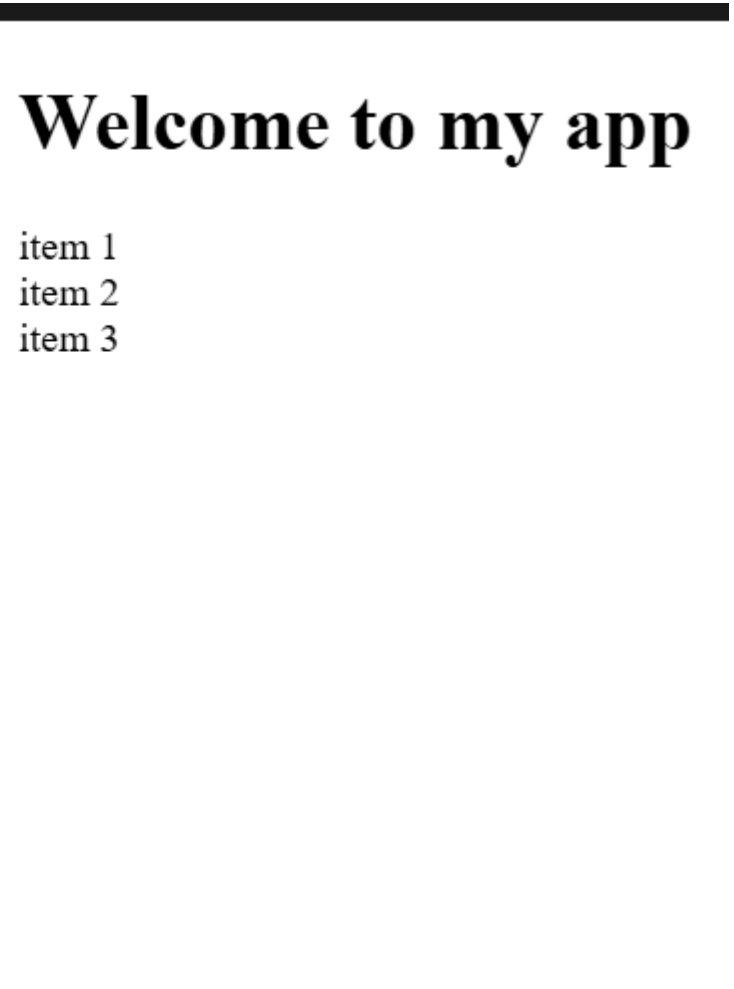
Let’s learn React – Displaying Data

Let’s try something more complex like displaying a list

Let’s learn React – Displaying Data

```jsx
function MyList() {
  const items = ["item 1", "item 2", "item 3"];
  return (
    <>
      {items.map((item) => (
        <li>{item}</li>
      ))}
    </>
  );
}

function App() {
  return (
    <div>
      <h1>Welcome to my app</h1>
      <MyList />
    </div>
  );
}
```



Let’s learn React – Displaying Data

We can even conditionally render components

Let’s learn React – Displaying Data

index.js

```
import React from "react";
import ReactDOM from "react-dom";

function MyButton() {···
}

function MyList() {···
}

function App() {
  const wantList = true;
  return (
    <div>
      <h1>Welcome to my app</h1>
      {wantList ? <MyList /> : <p>no list for you</p>}
    </div>
  );
}
```

https://g3jmcc.csb.app/

Welcome to my app

item 1
item 2
item 3

Let’s learn React – Updating the screen

- Often, you’ll want your component to “remember” some information and display it
- For example: in our to do app, that was the content of the list. In the calculator, that the value displayed on the screen
- To do this in React, we use **useState**
- You’ll get two things from useState: the current state (count), and the function that lets you update it (setCount). You can give them any names, but the convention is to write [something, setSomething]

index.js

src > index.js

```jsx
import React from "react";
import ReactDOM from "react-dom";
import { useState } from "react";

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return <button onClick={handleClick}>Clicked {count} times</button>;
}

function MyList() {…
}

function App() {
  const wantList = true;
  return (
    <div>
      <h1>Welcome to my app</h1>
      <MyButton />
    </div>
  );
}

```

Preview

https://g3jmcc.csb.app/

Welcome to my app

Clicked 9 times

Let’s learn React – Updating the screen

- **Recall:** Before, if we wanted to update the screen, we used to get the element and manually set its content. For example, your render function in the to do app or display in the calculator
- In React, we do this by **simply updating the state variable** and **react automatically handles updating the UI**
- We declared that the count variable should be displayed inside the button. When it’s updated, React automatically re-renders the component and therefore updates the screen
- This style for programming is called **Declarative Programming**. You declare what the end result should be, but you don’t worry how it will be achieved

Let’s learn React – Sharing data between components

- In the previous example, the count state/variable was inside the button component
- That means that no one can access it from outside the button
- What if we want to have 2 buttons, one for increment and one for decrement
- We use props… Props are basically passing arguments to the component
- Let’s see how we can achieve the counter increment/decrement

Each component is called with a parameter called “props”. This parameter will contain what ever you pass down when you call the component from the parent.

Here we included the count variable and a callback function to handle the button click

```
function PlusButton(props) {
  const count = props.count;
  const onClick = props.onClick;
  return <button onClick={onClick}>+ {count}</button>;
}

function MinusButton(props) {
  const count = props.count;
  const onClick = props.onClick;
  return <button onClick={onClick}>- {count}</button>;
}

function App() {
  const [count, setCount] = useState(0);

  function handlePlus() {
    setCount(count + 1);
  }

  function handleMinus() {
    setCount(count - 1);
  }

  return (
    <div>
      <PlusButton count={count} onClick={handlePlus} />
      <MinusButton count={count} onClick={handleMinus} />
    </div>
  );
}
```

You can destructure the received props like so ->

src > index.js

```jsx
import React from "react";
import ReactDOM from "react-dom";
import { useState } from "react";

function PlusButton({ count, onClick }) {
  return <button onClick={onClick}>+ {count}</button>;
}

function MinusButton({ count, onClick }) {
  return <button onClick={onClick}>- {count}</button>;
}

function App() {
  const [count, setCount] = useState(0);

  function handlePlus() {
    setCount(count + 1);
  }

  function handleMinus() {
    setCount(count - 1);
  }

  return (
    <div>
      <PlusButton count={count} onClick={handlePlus} />
      <MinusButton count={count} onClick={handleMinus} />
    </div>
  );
}
```

Why React?

Now let’s answer this question from what we understood:

1. React allows us to write components once and reuse them everywhere. Companies create their UI library of React components and frontend devs compose the UI from these components. Now that’s scalable!
2. Declarative programming: no need to worry about manually updating the UI
3. Huge ecosystem of libraries that can save you a lot of work

Lab task

Movies watch list: app for saving the movies that you watch along with their rating.

Requirements

1. Add, remove movies to the list
2. Adding a comment review to the movie
3. Add a starts rating to the movie.
4. Show how many starts you have given to the movie using emojies ⭐

You can use AI for styling but **NOT for React related work.**

Use the react docs.